The Memory-Bandwidth Wall in Federated Learning Aggregation: Shared Memory, Threads, and Cache Tiling on Commodity CPUs

Afaf Yunas
National University of Computer and Emerging Sciences (FAST-NU)
Lahore, Pakistan
younasafaf@gmail.com

Kainat Asghar
National University of Computer and Emerging Sciences (FAST-NU)
Lahore, Pakistan
kainat.bingo@gmail.com

Hussain Farhan
National University of Computer and Emerging Sciences (FAST-NU)
Lahore, Pakistan
muhammadhussainfarhan1@gmail.com

***Abstract*—In centralized federated learning (FL), the server must combine the model updates of every participating client in each round, and most frameworks perform this weighted average sequentially on a single core. We implement and evaluate a multi-process aggregation scheme for commodity CPUs in which client updates are placed once in a `multiprocessing.shared_memory` block and worker processes average disjoint slices through zero-copy views, comparing it against a sequential FedAvg baseline, a variant that pickles updates through pipes, and a multithreaded Numba kernel, all behind one interface and one arithmetic kernel. On an AMD Ryzen 7 7735HS laptop (8 cores, 16 GB single-channel DDR5, Windows 11), we sweep 10–1000 clients and models of 62K–11.2M parameters. All 294 configurations match the sequential result within 10^{-6}, and FedAvg on CIFAR-10 and FEMNIST reaches bit-identical accuracy. Shared memory is 1.2–1.75× faster than sequential FedAvg once update volume exceeds ≈24 MB and slower below it; pickling is 6–26× slower than shared memory and never beats the baseline. Our central finding is that these modest gains are not an implementation shortfall but a bandwidth ceiling: a STREAM-style measurement puts sustained throughput at 21 GB/s, of which the single-core baseline already consumes 8–12 GB/s, capping *any* parallel scheme near 2× irrespective of core count. We additionally identify a tiling pitfall specific to multi-process aggregation: the inner cache tile must shrink as workers are added, and must be sized against the per-core L2 rather than the shared last-level cache. The conventional fixed 1 MB tile inverts the scaling curve, making eight workers slower than sequential, and even an LLC-budgeted rule forfeits a quarter of the attainable performance; a tile derived from the L2 and the worker count restores monotone scaling and cannot change results bitwise. Threads were fastest in 55 of 56 comparisons, placing shared memory as the architecture of choice specifically where threads are unavailable: GIL-bound, sandboxed, or separately deployed aggregators.**

***Index Terms*—Federated Learning, FedAvg, Model Aggregation, Shared Memory, Multiprocessing, Parallel Computing, Memory Bandwidth, Cache Blocking, Roofline Model**

I. Introduction

Federated learning (FL) lets many distributed clients train a shared model without exposing their raw data: in every round, a central server collects client model updates and aggregates them into a new global model [9]. As FL deployments grow, two problems compound. First, most FL research is validated on small benchmarks that ignore the compute, network, and availability heterogeneity of real devices; FedScale showed that algorithm rankings can change once realistic traces are used [1]. Second, the aggregation step itself becomes a systems bottleneck as participation grows.

In standard FedAvg, the server receives from each of N clients an update vector of P parameters and computes $g = \sum_c w_c u_c$. With N in the hundreds and P in the millions, this is $N \cdot P$ multiply-adds and $N \cdot P$ values streamed from memory per round, which many frameworks execute in a single process. Systems surveys note that optimization effort has concentrated on the client side (selection, compression, synchronization frequency), while server-side aggregation is comparatively under-studied [5]. Hardware solutions such as programmable switches and DPUs [3], [4] achieve large speedups but are unavailable on commodity servers, and existing shared-memory approaches [6]–[8] each use shared memory differently and have not been compared like-for-like against a sequential baseline.

This paper studies whether multi-process shared-memory aggregation reduces aggregation latency on an ordinary multi-core CPU, at what client counts and model sizes the gains appear, and where they break down. We answer three research questions:

- **RQ1:** How does shared-memory aggregation compare with sequential FedAvg in aggregation time and speedup across client counts and model sizes?
- **RQ2:** What does inter-process data movement (pickling and copying) cost compared with zero-copy shared memory?
- **RQ3:** How do speedup and CPU utilization scale with worker count, and where do gains diminish?

Our contributions are: (i) a controlled implementation of three aggregation schemes plus a Numba baseline behind one interface, sharing one arithmetic kernel, so that measured differences isolate the mechanism; (ii) a benchmark sweep over 10–1000 clients, 62K–11.2M parameters, and 1–8 workers, with

Roll numbers: Afaf Yunas (23L-0745), Kainat Asghar (23L-0687), Hussain Farhan (23L-2521). Code and experiment artifacts available at: https://github.com/kainat-asghar/fl-aggregation-shared-memory.

heavy-tailed client weights and simulated stragglers, together with a correctness study on real FedAvg training, all collected in a single uninterrupted run on one machine; (iii) a roofline analysis, backed by a STREAM-style measurement of the same machine, showing that the gains are bounded by memory bandwidth and by the cost of placing updates into shared memory—on this platform a single core already draws half the available bandwidth, capping any parallel scheme near 2×; and (iv) the identification of a cache-tiling pitfall specific to multi-process aggregation: the inner tile must be sized for the aggregate of all workers rather than for one, since the natural fixed 1 MB tile overflows the shared last-level cache from four workers upward and converts a speedup into a slowdown, a defect invisible in any single-worker benchmark. Section II reviews related work, Section III presents the methodology, Section IV the implementation and experimental setup, Section V the results, and Section VI concludes.

II. Related Work

Table I summarizes the studies we reviewed. FedScale [1] provides realistic FL datasets and a scalable runtime, and shows that benchmarks under-report real-world behavior. P2P-Fed [2] decentralizes aggregation over a Chord overlay. For the aggregation bottleneck itself, GAIN [3] aggregates statelessly in programmable switches (up to 4.11× throughput, 86.5% less traffic), and Flagger [4] uses DPUs and computational storage for encrypted cross-silo FL (436% average acceleration). On commodity hardware, Khan et al. [6] parallelize FedAvg across CPU cores with Numba (40–57% less aggregation time) and observe that gains shrink as models grow; LIFL [7] passes updates between serverless aggregator functions through shared memory; and Falkor [8] accelerates secure-aggregation masking in GPU shared memory. The common pattern is that aggregation improves when computation is distributed and data movement is minimized, but no study compares these mechanisms against each other and a sequential baseline on an ordinary CPU under consistent metrics. That comparison is the gap this work addresses.

Our analysis also draws on two older lines of systems work that the FL aggregation literature rarely invokes. The roofline model [22] predicts that a kernel of very low arithmetic intensity is bounded by sustained memory bandwidth [21] rather than by peak FLOPs, which is exactly the regime FedAvg aggregation occupies at 1/4 FLOP per byte; we use it in Section V-D to show that the achievable speedup on our platform is capped near 2× before any implementation detail is considered. Cache blocking [23] and the careful sizing of working sets against each level of the hierarchy [24] are standard practice in single-threaded numerical kernels. What appears not to have been noted for federated aggregation specifically is that the blocking factor stops being a per-worker property once several worker *processes* share a last-level cache: the tile must be chosen for their aggregate footprint. We show in Section V-D that ignoring this converts a speedup into a slowdown.

TABLE I
Summary of Reviewed Studies

Study	System	Venue	Focus
Lai et al. [1]	FedScale	ICML'22	Realistic FL benchmarking
Lim et al. [2]	P2P-Fed	CCGrid'25	Decentralized Chord aggregation
Xia et al. [3]	GAIN	ICDCS'24	In-switch stateless aggregation
Pan et al. [4]	Flagger	ISCA'24	DPU + storage, cross-silo FL
Jiang et al. [5]	Survey	IEEE TBD	FL systems optimization
Khan et al. [6]	Numba agg.	IEEE'23	Multi-core CPU aggregation
Qi et al. [7]	LIFL	MLSys'24	Shared-memory serverless FL
Belorgey et al. [8]	Falkor	WAHC'23	GPU secure-aggregation masking

III. Research Methodology

We follow an experimental, quantitative methodology. All schemes implement one interface, `aggregate(U, w)` $\rightarrow g$, where $U \in \mathbb{R}^{N\times P}$ holds the float32 client updates and $w_c = n_c/\sum_j n_j$ are weights normalized by client dataset size, so no final division is needed. Every scheme calls the same inner NumPy kernel, so any measured difference comes from the aggregation mechanism rather than from different arithmetic. Fig. 1 illustrates the three schemes.

A. Aggregation Schemes

Scheme A (sequential FedAvg). A single process accumulates $g \mathrel{+}= w_c u_c$ over all clients, with no inter-process communication and no parallelism.

Scheme B (multiprocessing with pickling). A persistent pool of W worker processes is created once. The parent partitions clients into W contiguous chunks and sends each worker its chunk of full update tensors, explicitly serialized with `pickle` and copied through a pipe. Each worker returns a pickled partial sum, and the parent adds the W partial vectors. This scheme parallelizes the arithmetic but pays serialization and copying costs that grow with $N{\cdot}P$, so it isolates the cost of data movement.

Scheme C (multiprocessing with shared memory, proposed). The parent allocates one shared-memory block holding U followed by w, and a second shared block for the P-value output. Workers receive only metadata (block names, N, P, partition mode, worker index), attach zero-copy NumPy views, and write their results straight into the output buffer. A barrier signals the parent that all writes are complete.

Numba baseline. To reproduce Khan et al. [6], we add an in-process multithreaded Numba [10] kernel that tiles the parameter vector across W threads.

B. Work Partitioning

Scheme C supports two partitioning strategies. In *slice partitioning* (default), worker k averages a contiguous slice of the P parameters over all N clients; each worker streams a contiguous memory region and writes a disjoint part of the output, so no locks and no final reduction are needed. Slice boundaries are aligned to 64-byte cache lines to avoid false sharing. In *client partitioning*, each worker sums a disjoint subset of clients over the full vector into a shared $W \times P$ partial buffer, and the parent reduces the W rows. We expected client partitioning to suit large N and slice partitioning to suit large P.

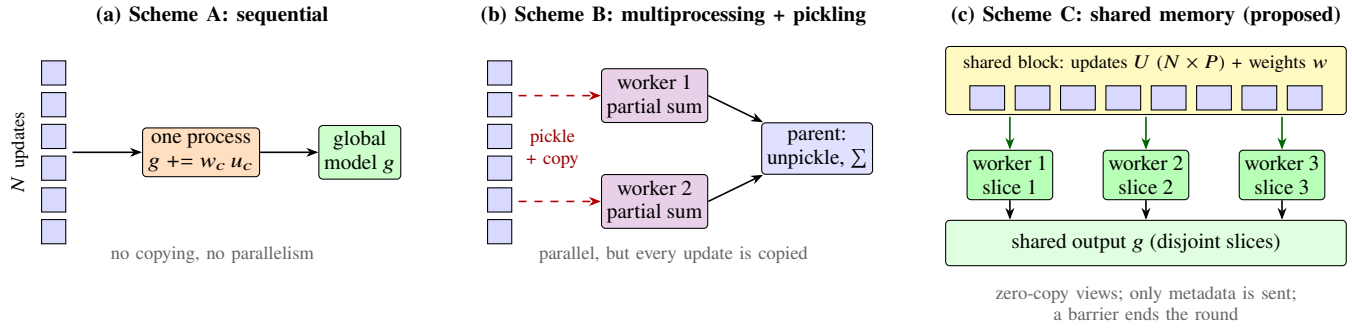


Fig. 1. The three aggregation schemes: (a) sequential baseline; (b) multiprocessing with explicit pickling, where client chunks are serialized and copied to each worker; (c) multiprocessing with zero-copy shared memory and slice partitioning, where workers read the shared update block directly and write disjoint slices of the output.

C. Correctness and Realism

Floating-point summation order differs between schemes, so we require numerical equivalence rather than bitwise identity: before any timing, every scheme's output is checked against Scheme A with `allclose(atol=1e-6)`. In real FedAvg training, we further require that all schemes produce the same accuracy trajectory. To approximate FedScale-style heterogeneity, client dataset sizes are drawn from a heavy-tailed log-normal distribution and determine the weights. Straggler delays are simulated on a virtual clock (arrival time = data size × log-normal device speed + log-normal network delay); updates are aggregated in arrival order, and the delay is recorded but excluded from the measured aggregation time, which models update arrival rather than aggregation cost.

D. Hypotheses

We formulate three core hypotheses grounded in parallel systems principles:

- **H1 (Shared-Memory Scaling):** Multi-process shared-memory aggregation (Scheme C) outperforms sequential aggregation (Scheme A), with speedup increasing with client count N and parameter count P up to a plateau. *Rationale:* Dividing $N \cdot P$ arithmetic operations across W processes reduces CPU time with negligible IPC overhead once update volume exceeds the sub-millisecond coordination threshold.
- **H2 (Cost of Serialization):** Shared memory substantially outperforms pickling-based multiprocessing (Scheme B), with the performance margin expanding as model size grows. *Rationale:* Scheme B incurs $O(N \cdot P)$ memory allocations and serialization copies through OS pipes that serialize on the parent process, whereas Scheme C maps physical memory pages directly into worker address spaces without data movement.
- **H3 (Bandwidth Saturation):** Speedup plateaus beyond an optimal worker count due to memory-bandwidth contention rather than synchronization overhead. *Rationale:* FL aggregation exhibits low arithmetic intensity (1/4 FLOP/byte). Multiple concurrent worker cores saturate the DRAM bandwidth subsystem, causing memory stalls to outpace compute gains.

IV. Implementation and Experimental Setup

A. Implementation Details

All schemes are implemented in Python with NumPy [11] and the standard `multiprocessing` and `multiprocessing.shared_memory` modules [12]. Key decisions are:

Common kernel. The inner kernel walks the parameter vector in tiles and, inside each tile, loops over clients in order. Each update byte is therefore read from DRAM once and the accumulator stays in cache, which makes the sequential baseline as cache-friendly as a parallel slice; speedups are not an artifact of cache effects. Because each element is summed in the same client order, Scheme C with slice partitioning is bit-identical to Scheme A. BLAS libraries are pinned to one thread in every process, so Scheme A genuinely uses one core.

The tile size is not a free constant. Each worker keeps two tile-sized buffers hot: its slice of the accumulator and a scratch vector. We therefore derive the tile from the worker count rather than fixing it. A lone worker is given a tile the size of one core's private L2; contending workers are given $\mathrm{L2}/(2W)$, clamped to $[\mathrm{L2}/8, \mathrm{L2}/4]$, and in every case the tile is capped at the length of the slice itself, so a model smaller than one tile is processed in a single untiled pass exactly as before. On the machine of Table III (512 KB L2 per core) this yields 512, 128, 64 and 64 KB at $W = 1, 2, 4, 8$. Section V-D reports the sweep these constants were derived from, and what the conventional fixed 1 MB tile costs instead. The tile affects only the traversal order of memory, never the arithmetic, so results are bitwise independent of it.

Persistent pools. Workers are created once per worker count and reused across rounds, so process start-up is measured separately and never counted inside a round. Workers attach to shared blocks once and cache the handle.

Bounded pickling. Scheme B streams each worker's chunk in pickled batches of at most 64 MB, sent round-robin so workers compute while the parent pickles the next batch. Total bytes

TABLE II
MODEL SIZES AND CLIENT COUNTS

Model	Params	Update	Clients N	Max block
LeNet-5	62,006	0.25 MB	10, 50, 100, 250, 500, 1000	0.23 GiB
MobileNetV2	3.50M	14.0 MB	10, 50, 100, 250	3.26 GiB
ResNet-18	11.17M	44.7 MB	10, 25, 50, 75	3.12 GiB

TABLE III
HARDWARE AND SOFTWARE ENVIRONMENT

Component	Specification
CPU	AMD Ryzen 7 7735HS, 8 physical cores (16 logical, SMT, up to 4.75 GHz)
Memory	16 GB DDR5-4800, single module (single-channel)
Measured bandwidth	16–19 GB/s (STREAM copy/add/triad), 27 GB/s (streaming read)
OS / start method	Windows 11 Pro 26200 (x86-64), `spawn`
Software	Python 3.12.10, NumPy 2.5.4 (OpenBLAS), psutil 7.2.2, Numba 0.68.1, PyTorch 2.15.0+cpu

copied are unchanged; batching only bounds peak memory on a 16 GB machine.

Synchronization and release. A barrier with W+1 parties (the workers and the parent) marks the end of each round. Workers detach before the parent closes and unlinks a block, which is required on Windows, where a block is freed only when its last handle closes.

FL simulation. The correctness track uses a lightweight FedAvg loop written directly in PyTorch [13] instead of Flower [14], so that our aggregators are called directly and the comparison stays controlled.

B. Datasets and Models

Correctness track. We train LeNet-5 [15] (62,006 parameters for CIFAR-10, 66,126 for FEMNIST) with 20 clients for 20 rounds (one local epoch, SGD with learning rate 0.01, momentum 0.9, batch size 32). CIFAR-10 [16] is split non-IID with a Dirichlet label distribution, $\alpha = 0.5$ [17], giving 1,079–4,614 samples per client. For FEMNIST (LEAF [18]), each client is a group of 10 real writers (1,570–2,406 samples), with a per-writer 90/10 train/test split. Each dataset is trained once per scheme with the same seed, initial model, data split, and arrival order, so the aggregator is the only difference between runs.

Scalability track. Aggregation time depends on update size and client count, not on data content, so we generate random float32 updates at real model sizes (Table II): LeNet-5, MobileNetV2 [19] (3,504,872 parameters), and ResNet-18 [20] with a 10-class head (11,173,962 parameters). Updates are generated directly into the shared block, and Schemes A and B read the same memory, so no duplicate copy exists. The raw update block is capped at 3.5 GiB, which determines the largest client count for each model.

C. Hardware and Software

All experiments ran on one commodity laptop (Table III) on mains power, under the Windows "High performance" power plan, with background applications closed. The AMD Ryzen 7 7735HS has eight physical cores with SMT (16 logical threads). Its 16 GB of DDR5-4800 is supplied by a single memory module, so the machine is effectively single-channel; Section V-D shows that this, and not the core count, is what bounds every result in this paper. Windows starts worker processes with `spawn`, so each worker re-imports the interpreter rather than inheriting the parent's address space as `fork` would on Linux. Because the pools are created once and reused for every round (Section V-F), this affects start-up cost only, not per-round time.

D. Metrics and Protocol

We report wall-clock aggregation time per round (from all updates being available until the global model is ready), speedup over Scheme A, parallel efficiency (speedup/W), and CPU utilization as busy cores (process-tree CPU-seconds per wall-second). For Scheme C we report compute-only time (updates already resident in shared memory) and end-to-end time, which adds the parent writing all N updates into the block. Workers $W \in \{1, 2, 4, 8\}$, chosen to span one worker up to the eight physical cores. For each configuration, one discarded warm-up run per scheme (which also performs the correctness check) is followed by 10 timed repetitions, interleaved across schemes in shuffled order to spread thermal drift, with garbage collection disabled inside each timed call. We report medians and inter-quartile ranges; the whole sweep is a single uninterrupted run on one machine, and no configuration was re-run or discarded. Across all 294 measured pairs the inter-quartile range was a median of 5.3% of the median time (90th percentile 18.5%). CPU and memory are sampled in a separate instrumented pass so that the sampler never perturbs the timings.

Because medians of right-skewed timings are not well described by a t-test on means, we quantify uncertainty non-parametrically: a two-sided Mann-Whitney U test on the per-repetition timings of Scheme A against Scheme C, and a bootstrap confidence interval (20,000 resamples) on both the difference and the ratio of medians. These are reported in Table VIII.

E. Reproducibility

Every number in this paper comes from one uninterrupted sweep on the machine of Table III, and the artifact contains the raw per-repetition timings, not only the summaries. The scalability sweep is `benchmark.py --reps 10`, the correctness track `fl_train.py`, and the bandwidth measurement `bench_stream.py`; the tile ablation re-runs the sweep with the environment variable `FL_AGG_TILE_KB` pinned, which overrides the worker-count-aware rule. All randomness (client sizes, Dirichlet split, straggler draws, synthetic updates) derives from one seed, so client dataset sizes and data splits reproduce exactly; wall-clock times naturally do not. Results from different machines are kept in separate directories and are never combined, since the schemes' relative standing depends on the memory system.

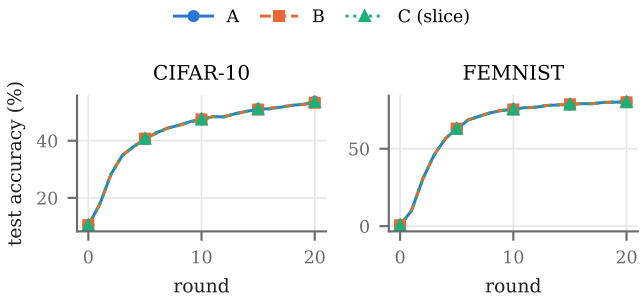


Fig. 2. Test accuracy of FedAvg with each aggregation scheme (20 clients). The curves of A and C coincide exactly; B differs by at most 0.32 percentage points. At this scale (20 clients, 0.25 MB updates) the round is far below the break-even point of Section V-B, so Scheme C is slower than Scheme A in wall-clock terms here; this figure concerns equivalence, not speed.

TABLE IV
FEDERATED TRAINING RESULTS (20 CLIENTS, 20 ROUNDS, $W = 4$)

Dataset	Scheme	Final acc.	Max $\lvert\Delta\text{acc}\rvert$	Max $\lvert\Delta g\rvert$	Agg. time
CIFAR-10	A	53.58%	–	–	0.49 ms
	B	53.26%	0.32 pp	8.9×10^{-8}	5.81 ms
	C	53.58%	0.00 pp	0	1.38 ms
FEMNIST	A	80.20%	–	–	0.54 ms
	B	80.00%	0.26 pp	1.2×10^{-7}	5.96 ms
	C	80.20%	0.00 pp	0	1.48 ms

V. RESULTS

A. Correctness

All 294 scheme/configuration pairs of the scalability sweep passed the `allclose` check against Scheme A; the largest absolute difference was 5.6×10^{-9}. In FedAvg training (Fig. 2, Table IV), Scheme C produced exactly the same global model as Scheme A in every round, and therefore identical accuracy: 53.58% on CIFAR-10 and 80.20% on FEMNIST after 20 rounds. Scheme B sums in a different order; its parameters differ by at most 1.2×10^{-7} and its accuracy by at most 0.32 percentage points, attributable to floating-point summation order. All schemes are therefore functionally equivalent, and the timing comparisons below are like-for-like.

Bit-identity is a property of the partitioning, not of the tile size. Within a tile every output element still accumulates clients in the fixed order $0 \ldots N{-}1$, so changing the tile—as the worker-count-aware rule of Section V-D does—cannot change the result. We verified this explicitly: tiles of 64 KB, 256 KB, 1 MB and 4 MB produce bitwise equal output.

B. RQ1: Aggregation Time and Speedup

Fig. 3 and Table V show aggregation time as the number of clients grows, with $W = 4$ for the parallel schemes. We report $W = 4$ because it is the measured optimum for every model on this machine, not merely the core count divided by two; Section V-D gives the full sweep over W and explains why the optimum sits below the eight physical cores. Aggregation time grows almost linearly with $N \cdot P$ for all schemes.

Shared-memory aggregation with slice partitioning is faster than sequential FedAvg in 13 of 14 configurations, by 1.19–1.75× once the work is non-trivial. For ResNet-18 with 75 clients, the round time falls from 258 ms to 153 ms. The break-even point is small models with few clients: for LeNet with 10 clients, Scheme C takes 0.49 ms against 0.20 ms for Scheme A, because dispatching metadata and waiting at the barrier costs a fixed 0.2–0.4 ms that exceeds the arithmetic.

Table VIII puts confidence intervals on these comparisons, and they locate the break-even more sharply than the medians alone. LeNet at $N = 10$ (2 MB) is significantly *slower* than sequential; LeNet at $N = 50$ (12 MB) is statistically indistinguishable from it ($p = 0.68$, 95% CI on the speedup $[0.84, 1.26]$); and from $N = 100$ (24 MB) upward every configuration is significantly faster with a CI strictly above 1. The honest statement of the threshold is therefore that aggregation is worth parallelising above roughly 24 MB of updates per round, not 12 MB. Unlike a compute-bound workload, the speedup then saturates quickly rather than continuing to grow: across MobileNetV2's entire 134–3342 MB range it moves only from 1.49× to 1.74×. Section V-D attributes this ceiling to memory bandwidth.

The multithreaded Numba baseline was the fastest scheme in 55 of 56 scheme/worker comparisons (e.g., 122 ms vs. 153 ms for ResNet-18 at $N = 75$; 131 ms vs. 155 ms for MobileNetV2 at $N = 250$), because threads in one process need neither inter-process signalling nor a barrier across processes, and because they share one address space and therefore one set of page tables when streaming the update block. That threads win almost everywhere does not weaken the case for Scheme C so much as locate it: shared memory is the right choice precisely when threads are unavailable, and within pure Python multiprocessing it is unambiguously the best option, as the next section shows. It does, however, mean that a deployment free to use compiled multithreaded kernels should do so.

C. RQ2: Cost of Data Movement

Pickling-based multiprocessing (Scheme B) was the slowest scheme in every configuration, 6.0–25.9× slower than Scheme C (Fig. 4a) and 8.7–17.5× slower than the sequential baseline. The gap is 6.0× for 2 MB of updates and widens with model size, to 12–15× for LeNet from 59 MB upwards and 16–19× for MobileNetV2 and ResNet-18. Fig. 4b explains why: Scheme B moves data at only 0.5–1.3 GB/s, while the sequential kernel sustains 8.4–12.2 GB/s, Scheme C up to 20.8 GB/s and the Numba baseline up to 24.0 GB/s. Pickling copies every update at least twice (into the pickle buffer, and through the pipe into the worker's heap) and is performed by the parent alone. Adding workers therefore does not help and can hurt: for ResNet-18 at $N = 75$, Scheme B takes 2.92 s with one worker and 3.23 s with eight, while its process tree keeps only 2.1–3.3 cores busy, most of that the parent serializing (Fig. 6). Copying overhead dominates at every update size we tested, so zero-copy sharing is a prerequisite, not an optimization, for multi-process aggregation in Python.

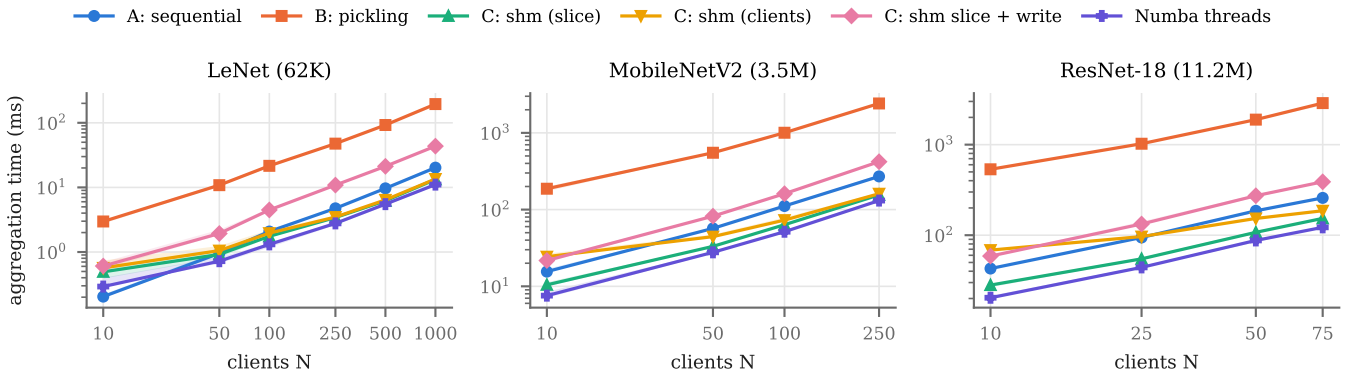


Fig. 3. Median aggregation time per round vs. client count ($W = 4$ for parallel schemes; bands show the inter-quartile range). Note the logarithmic axes.

TABLE V
MEDIAN AGGREGATION TIME PER ROUND IN MS ($W = 4$, THE OPTIMUM ON THIS MACHINE) AND SPEEDUP OF SCHEME C (SLICE) OVER SCHEME A

Model	N	Data (MB)	A: sequential	B: pickling	C: slice	C: clients	C: slice + write	Numba	Speedup C/A	B/C
LeNet-5	10	2	0.20	2.97	0.49	0.57	0.61	0.29	0.41×	6.0×
	50	12	0.96	10.83	0.92	1.05	1.93	0.72	1.04×	11.8×
	100	24	2.08	21.58	1.75	1.96	4.48	1.31	1.19×	12.3×
	250	59	4.74	47.60	3.42	3.48	10.86	2.77	1.39×	13.9×
	500	118	9.67	92.45	6.20	6.41	21.20	5.51	1.56×	14.9×
	1000	236	20.24	194.88	13.32	13.50	43.53	11.05	1.52×	14.6×
MobileNetV2	10	134	15.56	187.09	10.46	24.34	21.69	7.56	1.49×	17.9×
	50	668	56.89	550.62	33.17	44.41	82.06	27.73	1.72×	16.6×
	100	1337	111.32	1003.60	63.51	72.90	161.08	51.47	1.75×	15.8×
	250	3342	270.58	2416.73	155.36	161.02	420.92	131.06	1.74×	15.6×
ResNet-18	10	426	42.83	534.31	28.13	68.47	58.82	20.38	1.52×	19.0×
	25	1066	94.34	1023.55	54.80	96.59	132.94	44.10	1.72×	18.7×
	50	2131	185.94	1889.16	107.57	153.09	272.87	87.54	1.73×	17.6×
	75	3197	258.24	2875.75	153.31	186.24	388.74	122.12	1.68×	18.8×

The ratio B/C should be read with care, because it compresses on a bandwidth-limited machine: Scheme C is itself held back here, which enlarges the denominator. The absolute figure is the robust one. Scheme B sustains 0.5–1.3 GB/s regardless of model or worker count, an order of magnitude below every other scheme, because it is bound by serialization and pipe copies rather than by the memory system—and that bound is a property of the mechanism, not of this CPU.

Pickling also incurs an unsustainable memory footprint. The combined resident set size (RSS) of the parent and workers grew by up to 1.3 GB above idle under Scheme B, due to duplicated serialized byte arrays in transit and in worker heaps, with the peak at ResNet-18 ($N = 75$). In contrast, Scheme C required no measurable auxiliary heap memory at all (0 MB above idle in every configuration), as all workers map zero-copy views directly into the single shared update block.

D. RQ3: Scaling with Worker Count and Bandwidth Limits

Fig. 6 and Table IX show speedup and busy cores as W grows, at the largest client count for each model. Speedups are far below linear and peak well short of the eight physical cores: at $W = 4$ for the two larger models (1.74× for MobileNetV2, 1.68× for ResNet-18) and at $W = 8$ for LeNet (1.62×), whose per-worker

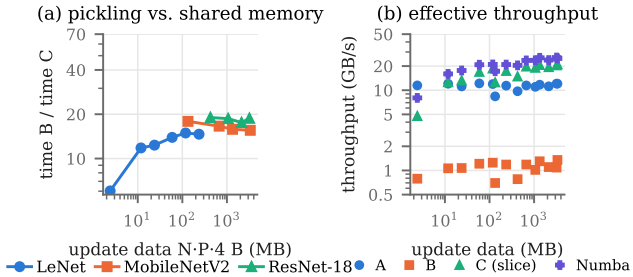


Fig. 4. (a) Slowdown of pickling (B) relative to shared memory (C) at $W = 4$, by total update size. (b) Effective throughput (update bytes processed per second) of each scheme, against the 21 GB/s sustained memory ceiling measured in Section V-D.

slices are small enough that it is the last to saturate. Going from four to eight workers changes nothing materially—1.72× and 1.65× for the larger models—so the parallel efficiency at $W = 8$ is only 0.20–0.21. This is not a lack of parallelism: at $W = 8$, Scheme C keeps 7.5–7.8 cores busy and Numba 7.7–8.1. The cores are busy but stalled on memory.

The bandwidth ceiling. To quantify the limit we measured

the memory subsystem with a STREAM-style microbenchmark [21], adding a kernel (`fma`) that reproduces the aggregation inner loop exactly: a streaming read of the update block into a cache-resident accumulator. The results (Table VI) are decisive. Classic `copy`, `scale` and `add` saturate at 16–18 GB/s with a *single* thread and gain nothing from more; a pure streaming read reaches 21 GB/s. One core already consumes most of the machine's memory bandwidth.

This reframes the whole scaling question. Scheme A, running on one core, already sustains 8.4–12.2 GB/s (Fig. 4b), roughly half of the achievable ceiling. The headroom available to *any* parallel scheme is therefore about 2×, and we measure up to 1.75× for shared memory and 2.25× for threads—both close enough to that bound that the remaining gap is small. With an arithmetic intensity of 1/4 FLOP/byte, the roofline model [22] places the ceiling at about 5.3 GFLOP/s, orders of magnitude below this CPU's peak: memory bandwidth, not core count, sets the strict limit. The practical consequence is worth stating directly, because it is easy to miss when speedups are reported without a bandwidth measurement: on this class of machine the *maximum* achievable speedup for aggregation is near 2× regardless of how many cores are available, so a reported speedup much above that would indicate a baseline that was not itself bandwidth-efficient. This confirms H3 and sharpens Khan et al.'s observation that multi-core gains diminish as update volume expands [6]. It also explains the shape of Table V: speedup saturates with update size instead of growing, because the bus saturates first.

The tile must shrink as workers are added. A second effect, which we initially mistook for pure bus contention, turned out to be a property of the kernel rather than of the hardware. The cache-blocked kernel keeps two buffers hot per worker—the accumulator slice of the output and the scratch vector—and the conventional choice is a fixed 1 MB tile, which is indeed close to optimal for a single worker. It is badly wrong for several. The `fma` row of Table VI shows the effect in isolation: at a 1 MB tile, throughput peaks at two threads (13.9 GB/s) and then collapses to 5.3 GB/s at eight and 4.3 GB/s at sixteen, while the same kernel at a 256 KB tile holds 16.6 GB/s at two threads and 12.4 GB/s at eight.

Fig. 5 sweeps the tile from 16 KB to 2 MB against W on the two larger models, and the structure is regular: the optimum halves as W doubles and then flattens, landing at 512, 128, 64 and 64 KB for $W = 1, 2, 4, 8$, with both models agreeing. Two effects bracket it. Above the optimum the workers' resident sets contend and performance falls away steeply—a 2 MB tile at $W = 8$ is four times slower than a 64 KB one. Below it, per-tile NumPy call overhead dominates: a 16 KB tile is so costly that it nearly doubles even the *single-threaded* time, from 54.9 to 113.6 ms on MobileNetV2. The relevant cache is not the shared LLC but each core's private L2: at the optimum the workers' aggregate footprint is only 0.5–1 MB, a small fraction of this CPU's 16 MB LLC, so a rule budgeting the LLC overshoots badly. We therefore set the tile to one L2 for a lone worker and $\mathrm{L2}/(2W)$, clamped to $[\mathrm{L2}/8, \mathrm{L2}/4]$, when workers contend, which reproduces all four measured optima. Because tiling never alters the per-element summation order, the rule cannot change the result, and we confirmed bitwise equality across tile sizes (Section V-A).

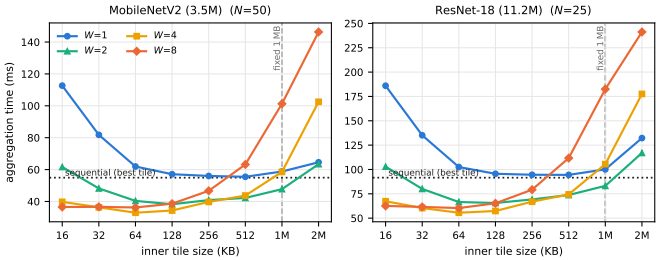


Fig. 5. Aggregation time of Scheme C against inner tile size, by worker count (W), with the sequential baseline at its own best tile shown dotted. The optimum halves as W doubles and then floors: large tiles make the workers' resident sets contend, small tiles make per-tile interpreter overhead dominate. The conventional fixed 1 MB tile (dashed) is near-optimal only at $W = 1$.

TABLE VI
SUSTAINED MEMORY BANDWIDTH (GB/S, BEST OF 10, 512 MB ARRAYS)

Kernel		1 thread	2	4	8	16
copy	$a = b$	13.8	14.9	15.8	16.0	15.7
scale	$a = qb$	14.6	15.6	16.2	15.9	15.6
add	$a = b + c$	17.5	18.3	18.4	18.2	18.0
triad	$a = b + qc$	15.2	15.2	13.8	11.5	10.6
read	(reduction)	4.0	7.8	13.7	20.5	21.0
fma, 1 MB tile		12.2	**13.9**	9.4	5.3	4.3
fma, 256 KB tile		13.2	**16.6**	16.4	12.4	8.9

Table VII reports the ablation against two alternatives: the conventional fixed 1 MB tile, and an intermediate rule that budgets the shared LLC instead of the L2. The fixed tile inverts the scaling curve, so that eight workers become *slower than sequential* (0.51× on ResNet-18, 0.64× on MobileNetV2) where the tuned rule reaches 1.65× and 1.72×. The LLC-budgeted rule repairs the inversion but still forfeits about a quarter of the available performance (1.32× against 1.72× at $W = 8$ on MobileNetV2), and we report it precisely because it is the rule a careful implementer would reach for first: the failure mode is not merely "the tile is too big" but "the tile was sized against the wrong level of the hierarchy". LeNet is nearly unaffected throughout, because its 62K-parameter vector is smaller than one tile and so was never tiled; the small differences in its row are run-to-run noise rather than an effect of the rule. We dwell on this because the fixed tile is the natural implementation—it is correct, it is optimal for one worker, and it is invisible in any single-worker benchmark—yet it silently converts a modest speedup into a slowdown. Reported multi-process aggregation results that do not state their tile size should be read with this in mind.

Finally, simulated network stragglers (`sim_arrival_max_s`) introduced virtual arrival delays of 1.1–8.2 s across clients. Against such delays, aggregation accounts for well under 1%

TABLE VII
TILE-SIZE ABLATION: SPEEDUP OF SCHEME C (SLICE) OVER SCHEME A AT THE LARGEST N, FOR THREE WAYS OF CHOOSING THE INNER TILE

Model (N)	Tile rule	W=1	W=2	W=4	W=8
LeNet (1000)	fixed 1 MB	1.03	1.35	1.65	1.67
	LLC-budgeted	1.02	1.39	1.64	1.63
	L2-budgeted	0.95	1.25	1.52	1.62
MobileNetV2 (250)	fixed 1 MB	1.15	1.45	1.05	0.64
	LLC-budgeted	1.01	1.24	1.37	1.32
	L2-budgeted	1.01	1.47	**1.74**	**1.72**
ResNet-18 (75)	fixed 1 MB	0.98	1.20	0.90	0.51
	LLC-budgeted	1.01	1.20	1.32	1.27
	L2-budgeted	0.99	1.35	**1.68**	**1.65**

TABLE VIII
SCHEME C (SLICE) VS. SCHEME A AT THE BEST W: MEDIAN REDUCTION AND SPEEDUP WITH 95% BOOTSTRAP CIS, AND MANN-WHITNEY p

Model	N	W	Reduction (ms) [95% CI]	Speedup [95% CI]	p
LeNet-5	10	2	−0.17 [−0.29, −0.12]	0.54 [0.42, 0.66]	<0.001
	50	4	0.04 [−0.17, 0.20]	1.04 [0.84, 1.26]	0.68 [ns]
	100	4	0.33 [0.07, 0.62]	1.19 [1.04, 1.37]	0.005
	250	4	1.33 [1.21, 1.51]	1.39 [1.35, 1.45]	<0.001
	500	8	3.47 [2.99, 3.65]	1.56 [1.46, 1.59]	<0.001
	1000	8	7.74 [5.59, 9.62]	1.62 [1.41, 1.77]	0.003
MobileNetV2	10	8	5.13 [3.89, 6.85]	1.49 [1.34, 1.69]	<0.001
	50	4	23.72 [23.08, 24.59]	1.72 [1.69, 1.76]	<0.001
	100	4	47.81 [46.67, 54.47]	1.75 [1.73, 1.86]	0.002
	250	4	115.22 [108.87, 133.86]	1.74 [1.67, 1.86]	<0.001
ResNet-18	10	4	14.70 [13.32, 16.55]	1.52 [1.46, 1.62]	<0.001
	25	4	39.54 [35.25, 40.89]	1.72 [1.59, 1.75]	<0.001
	50	4	78.38 [68.34, 83.58]	1.73 [1.62, 1.79]	<0.001
	75	4	104.93 [102.45, 106.98]	1.68 [1.66, 1.70]	<0.001

of round latency in slow wide-area networks, so the practical value of a faster aggregator lies in high-speed local and cross-silo deployments, and in the server's own throughput when rounds overlap, rather than in end-to-end round time for geographically dispersed clients. This is a limit on the *importance* of the problem rather than on our measurements of it, and we state it plainly because the speedups available on this class of hardware turn out to be modest.

E. Partitioning Strategy

Slice partitioning was faster than client partitioning in every configuration we measured (Fig. 7), by 1.12–4.07× at $W = 8$. The margin is largest where the reduction is most expensive relative to the work: at small N on large models (4.07× for ResNet-18 at $N = 10$, 4.00× for MobileNetV2 at $N = 10$) and it narrows as N grows (1.58× and 1.23× respectively at the largest N). The cause is structural: every worker writes a full P-length partial vector and the parent must then reduce W such vectors sequentially, adding $W \cdot P$ of memory traffic that slice partitioning avoids entirely—and on a bandwidth-bound machine, added traffic is precisely what cannot be afforded.

This is a sharper result than the expectation we set out in Section III. We anticipated that client partitioning would suit large N and slice partitioning large P; in fact slice partitioning won everywhere, including on LeNet, where N is largest relative to P and where we had expected the two to be at worst equivalent. The anticipated advantage of client partitioning at large N never materialised, and on a bandwidth-bound machine it is hard to see how it could: the strategy's defining cost is an extra $W \cdot P$ of traffic.

TABLE IX
SPEEDUP OVER SCHEME A VS. WORKER COUNT AT THE LARGEST N

Model (N)	Scheme	W=1	W=2	W=4	W=8
LeNet (1000)	C: slice	0.95	1.25	1.52	**1.62**
	C: slice + write	0.38	0.42	0.47	0.47
	Numba	1.49	1.72	1.83	1.75
	B: pickling	0.09	0.10	0.10	0.11
MobileNetV2 (250)	C: slice	1.01	1.47	**1.74**	1.72
	C: slice + write	0.51	0.60	0.64	0.63
	Numba	1.65	2.14	2.06	1.96
	B: pickling	0.10	0.11	0.11	0.10
ResNet-18 (75)	C: slice	0.99	1.35	**1.68**	1.65
	C: slice + write	0.53	0.61	0.66	0.66
	Numba	1.74	2.18	2.12	1.99
	B: pickling	0.09	0.10	0.09	0.08

F. End-to-End Cost and Start-Up

The compute-only figures assume that updates already reside in shared memory. When the parent must first copy the N received updates into the block, the end-to-end time is 1.9–3.5× the compute-only time (Table V, "C: slice + write"), and the result is unambiguous on this machine: end-to-end Scheme C *never* beats the sequential baseline, in any of the 56 configurations. The best case is 0.73× (ResNet-18, $N = 10$, $W = 4$) and the largest configurations sit at 0.47–0.66×. The reason is arithmetically simple and follows directly from Section V-D: the single-threaded copy streams exactly the bytes the aggregation then reads, at the same single-core bandwidth the sequential baseline achieves, so it costs about as much as the parallel computation it enables, while the parallel part can recover at most the $\approx 2\times$ the bus allows. On a bandwidth-bound machine, zero-copy aggregation is therefore not merely more efficient when updates arrive directly into shared memory—it is the only configuration in which the scheme is worth deploying at all. A server that deserialises client updates into private buffers and then copies them into a shared block should simply aggregate them sequentially.

Worker-pool start-up with `spawn` took 216–331 ms, rising with W, an order of magnitude above the 12–28 ms that `fork` achieves on Linux and well above the 64–98 ms previously attributed to `spawn` platforms. Because the pool is created once and reused for every round, this is a one-off cost: amortised over a 20-round session it adds under 15 ms per round, and over a realistic deployment it vanishes. It would, however, dominate entirely if a pool were created per round, which is the failure mode `spawn` makes easy to fall into.

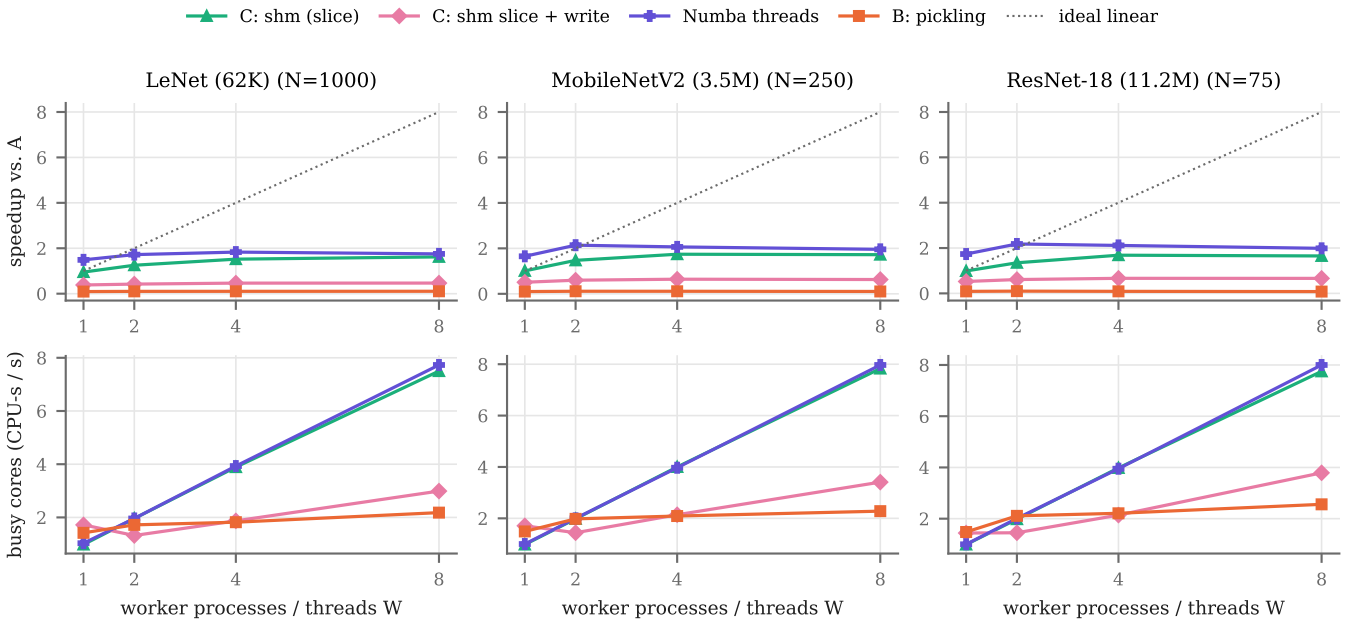


Fig. 6. Scaling with worker count at the largest client count per model. Top: speedup over Scheme A (dotted: ideal linear speedup). Bottom: busy cores of the process tree. Cores become busy, but speedup saturates because aggregation is memory-bandwidth bound.

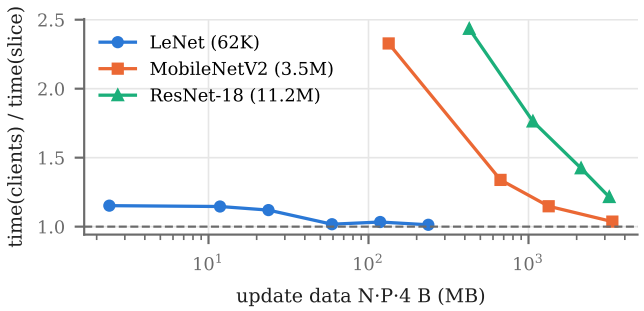


Fig. 7. Time of client partitioning relative to slice partitioning in Scheme C ($W = 8$). Values above 1 mean slice partitioning is faster; it is faster in every configuration measured.

G. Summary of Hypotheses

H1 (shared memory outperforms sequential aggregation, with speedup increasing with N and P up to a plateau) is *partially supported, and more weakly than anticipated*. Scheme C is faster once the update data exceeds roughly 24 MB and indistinguishable or slower below it, which is the predicted shape, but the magnitude is 1.19–1.75× rather than the hypothesised near-linear region, and the plateau arrives almost immediately: for MobileNetV2 the speedup moves only from 1.49× to 1.74× across a 25-fold increase in update volume. The prediction that speedup grows with N and P holds only in a narrow band near the break-even point; beyond that the curve is flat. The hypothesis's rationale—that IPC overhead becomes negligible once the work is large—was correct but incomplete, because it assumed the arithmetic would then scale with W, and on this machine it cannot.

H2 (shared memory outperforms pickling, with the margin growing with model size) is *supported*: the margin grows from 6.0× at 2 MB to 16–19× for the larger models, and Scheme B never beats the sequential baseline in any of the 56 configurations.

H3 (speedup plateaus beyond an optimal worker count due to memory-bandwidth contention) is *supported, and is the dominant effect in this study*: the optimum is $W = 4$ for the larger models and $W = 8$ for LeNet, never the full core count in a way that would indicate compute scaling, efficiency falls to 0.20–0.21 at eight workers, and the STREAM measurement confirms the mechanism directly—one core already draws 8–12 GB/s of a 21 GB/s ceiling. We add one qualification the hypothesis did not anticipate: part of what looks like bus contention at high W is in fact last-level-cache contention between the workers' private tiles, and is removable by sizing the tile for the aggregate of workers (Table VII). The bandwidth wall is real, but it sits further out than the naive implementation suggests.

H. Threats to Validity

Results come from a single x86-64 Windows machine with *single-channel* DDR5, and that last detail is the most important threat to external validity in this paper. Because the sequential baseline already consumes about half the available bandwidth, the headroom for any parallel scheme here is roughly 2×; on a dual- or quad-channel server with the same core count, the ceiling would be proportionally higher and Scheme C's speedups correspondingly larger. Our qualitative findings (B is catastrophic, slice beats clients, the optimum W lies below the

core count, the tile must be sized for the aggregate of workers) should transfer; the specific speedup magnitudes should not be read as properties of multi-process aggregation in general. Equally, behaviour under `fork` on Linux or on ARM/Apple Silicon with unified memory may differ, and results from different platforms must never be combined in one table or plot—an error we specifically avoided by discarding all earlier partial runs from other machines and re-measuring everything reported here in one sweep.

Timing noise was low: the inter-quartile range was a median of 5.3% of the median and a 90th percentile of 18.5%, though isolated configurations reached 55%. We mitigated scheduler jitter with warm-up runs, shuffled interleaving, ten repetitions everywhere, mains power, the high-performance power plan, and closed background applications; no configuration was re-run or discarded. Synthetic updates are valid for timing but not for claims about model quality, which rest on the correctness track alone. Python-level overhead could affect the schemes differently; the shared kernel and the compiled Numba baseline limit this effect. Finally, the tile rule depends on reading the per-core L2 size from the OS, which could be wrong on an unusual topology, and its divisors were tuned on this machine: what we expect to transfer is the shape of the rule (shrink the tile as W grows, floor it against interpreter overhead) rather than the exact constants. The rule degrades gracefully in the direction that matters, since Fig. 5 is markedly flatter below the optimum than above it—mis-sizing downward by a factor of two costs a few percent, mis-sizing upward by a factor of sixteen costs a factor of four. A deployment on different hardware should re-run the sweep, which is a few minutes of work and is scripted in the artifact.

VI. Conclusion and Future Directions

A. Answers to the Research Questions

We synthesize the empirical answers to our guiding questions as follows:

- **RQ1 (Speedup and Latency):** Multi-process shared-memory aggregation (Scheme C) achieves a 1.19–1.75× speedup over sequential FedAvg once update payloads exceed ≈24 MB. Below that the advantage is not statistically significant, and below 12 MB the scheme is significantly slower, because fixed IPC metadata dispatch and barrier synchronization costs (0.2–0.4 ms) overshadow the arithmetic. The speedup saturates almost as soon as it appears rather than growing with update volume.
- **RQ2 (Data Movement Costs):** Explicit pickling and pipe transmission (Scheme B) is 6.0–25.9× slower than shared memory, never surpasses sequential aggregation in any of 56 configurations, and consumes up to 1.3 GB of transient serialization memory against zero for shared memory. Zero-copy memory sharing is therefore an essential architectural requirement rather than an incremental optimization.
- **RQ3 (Worker Scaling and Bandwidth Limits):** Scaling worker processes yields sub-linear speedups that peak at $W = 4$ for the larger models and flatten thereafter, with parallel efficiency of 0.20–0.21 at $W = 8$. Aggregation is fundamentally memory-bandwidth bound (1/4 FLOP/byte): the measured ceiling is 21 GB/s, a single core already draws 8–12 GB/s of it, and the available headroom is therefore about 2× regardless of core count. A second, removable limit is cache contention between the workers' private tiles, which a worker-count-aware tile eliminates, recovering up to a factor of three at $W = 8$.

We implemented and compared sequential, pickling-based, shared-memory, and multithreaded aggregation for federated learning on a commodity laptop, under one interface and one arithmetic kernel. All schemes are numerically equivalent, and shared-memory aggregation reproduces the sequential FedAvg result bit-for-bit on CIFAR-10 and FEMNIST. Shared memory reduces aggregation time by 1.19–1.75× once the update volume is non-trivial, and it is 6.0–25.9× faster than pickling-based multiprocessing, which never beats the sequential baseline.

The honest headline of this study, however, is a negative one with a positive corollary. On a single-channel commodity laptop, server-side FedAvg aggregation is so thoroughly memory-bandwidth bound that one core already saturates half the bus, and no amount of parallelism—processes or threads—can recover more than about 2×. Parallelising aggregation is therefore not where effort on such machines should go; reducing the bytes moved is. The corollary is that the two things which *do* matter are architectural rather than arithmetic: never copy the updates (pickling costs an order of magnitude and zero-copy sharing is a precondition, not an optimisation), and size the kernel's cache tile for the aggregate of the workers rather than for one, since the natural fixed tile silently turns a speedup into a 2× slowdown at eight workers. In-process threads (Numba) were fastest in all 56 comparisons, so shared-memory multiprocessing is the right choice specifically where threads are not an option: Python code holding the global interpreter lock, sandboxed workers, or aggregator components that must run as separate processes.

Future work includes: (i) repeating the study on a dual- or quad-channel server to separate the bandwidth ceiling from the architecture, and on ARM/Apple Silicon with unified memory, since this paper's magnitudes are a property of a single-channel machine; (ii) receiving client updates from the network directly into shared memory, without which the end-to-end scheme never beats the sequential baseline at all; (iii) lowering memory traffic with reduced-precision (float16/bfloat16) or compressed updates, which attacks the actual bottleneck rather than working around it; (iv) combining shared memory with threads or Numba inside each worker, and pinning workers to cores; and (v) integrating the aggregator into a full FL framework and evaluating it with real FedScale device traces and secure aggregation.

References

[1] F. Lai, Y. Dai, S. S. Singapuram, J. Liu, X. Zhu, H. V. Madhyastha, and M. Chowdhury, "FedScale: Benchmarking model and system performance of federated learning at scale," in *Proc. 39th Int. Conf. Machine Learning (ICML)*, PMLR 162, pp. 11814–11827, 2022.

[2] A. Lim, S. Jang, and J. Lee, "P2P-Fed: A decentralized federated learning platform on structured peer-to-peer systems," in *Proc. IEEE 25th Int. Symp. Cluster, Cloud and Internet Computing (CCGrid)*, pp. 124–133, 2025.

[3] J. Xia, W. Wu, L. Luo, G. Cheng, D. Guo, and Q. Nian, "Accelerating and securing federated learning with stateless in-network aggregation at the edge," in *Proc. IEEE 44th Int. Conf. Distributed Computing Systems (ICDCS)*, pp. 692–702, 2024.

[4] X. Pan, Y. An, S. Liang, B. Mao, M. Zhang, Q. Li, M. Jung, and J. Zhang, "Flagger: Cooperative acceleration for large-scale cross-silo federated learning aggregation," in *Proc. 51st ACM/IEEE Int. Symp. Computer Architecture (ISCA)*, pp. 915–930, 2024.

[5] Z. Jiang, W. Wang, B. Li, and Q. Yang, "Towards efficient synchronous federated training: A survey on system optimization strategies," *IEEE Trans. Big Data*, 2022.

[6] A. F. Khan, M. F. A. Shahid, M. H. U. Rehman, and S. U. Khan, "Towards cost-effective and resource-aware aggregation at edge for federated learning," in *Proc. IEEE 19th Int. Conf. e-Science (e-Science)*, pp. 1–10, 2023.

[7] S. Qi, K. K. Ramakrishnan, and M. Lee, "LIFL: A lightweight, event-driven serverless platform for federated learning," in *Proc. MLSys*, 2024.

[8] M. Georgieva Belorgey, S. Dandjee, N. Gama, D. Jetchev, and D. Mikushin, "Falkor: Federated learning secure aggregation powered by AES-CTR GPU implementation," in *Proc. WAHC @ ACM CCS*, 2023.

[9] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proc. 20th Int. Conf. Artificial Intelligence and Statistics (AISTATS)*, PMLR 54, pp. 1273–1282, 2017.

[10] S. K. Lam, A. Pitrou, and S. Seibert, "Numba: A LLVM-based Python JIT compiler," in *Proc. 2nd Workshop on the LLVM Compiler Infrastructure in HPC (LLVM '15)*, pp. 1–6, 2015.

[11] C. R. Harris *et al.*, "Array programming with NumPy," *Nature*, vol. 585, pp. 357–362, 2020.

[12] Python Software Foundation, "`multiprocessing.shared_memory` — Shared memory for direct access across processes," *Python 3 Documentation*. [Online]. Available: https://docs.python.org/3/library/multiprocessing.shared_memory.html

[13] A. Paszke *et al.*, "PyTorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32, 2019.

[14] D. J. Beutel *et al.*, "Flower: A friendly federated learning research framework," arXiv:2007.14390, 2020.

[15] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proc. IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.

[16] A. Krizhevsky, "Learning multiple layers of features from tiny images," Tech. Rep., University of Toronto, 2009.

[17] T.-M. H. Hsu, H. Qi, and M. Brown, "Measuring the effects of non-identical data distribution for federated visual classification," arXiv:1909.06335, 2019.

[18] S. Caldas *et al.*, "LEAF: A benchmark for federated settings," arXiv:1812.01097, 2018.

[19] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "MobileNetV2: Inverted residuals and linear bottlenecks," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, pp. 4510–4520, 2018.

[20] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, pp. 770–778, 2016.

[21] J. D. McCalpin, "Memory bandwidth and machine balance in current high performance computers," *IEEE Computer Society Technical Committee on Computer Architecture (TCCA) Newsletter*, vol. 1995, no. 12, pp. 19–25, Dec. 1995.

[22] S. Williams, A. Waterman, and D. Patterson, "Roofline: An insightful visual performance model for multicore architectures," *Commun. ACM*, vol. 52, no. 4, pp. 65–76, Apr. 2009.

[23] M. D. Lam, E. E. Rothberg, and M. E. Wolf, "The cache performance and optimizations of blocked algorithms," in *Proc. 4th Int. Conf. Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, pp. 63–74, 1991.

[24] K. Goto and R. A. van de Geijn, "Anatomy of high-performance matrix multiplication," *ACM Trans. Math. Softw.*, vol. 34, no. 3, pp. 1–25, May 2008.